

Bank Management System

Complete Source Code & Project Documentation
Core Java Backend, Web Server & HTML5/CSS3/JS Web Application

Project Title:	Bank Management System Website & CLI
Technologies:	Core Java (JDK), HttpServer, REST APIs, HTML5, CSS3, JavaScript
Target Audience:	2nd-Year Engineering Students (Java & Web Dev)
Architecture:	Client-Server Architecture (REST API + Persistence)
Port:	http://localhost:8080

Generated from Source Code Repository • Apex National Bank Project

Table of Contents

PART 1: JAVA BACKEND & WEB SERVER

1.1 Account.java — Account entity model with balance & PIN methods

1.2 Transaction.java — Financial transaction record model

1.3 Bank.java — Core business logic engine & transaction processing

1.4 FileHandler.java — Text persistence layer for accounts.txt

1.5 Main.java — Interactive Console CLI Interface

1.6 BankWebServer.java — Lightweight HTTP Server with 7 REST API endpoints

PART 2: WEB FRONTEND (HTML5 / CSS3 / JAVASCRIPT)

2.1 index.html — Home landing page showcasing bank features

2.2 login.html — Customer login page with PIN authentication

2.3 register.html — New account registration page

2.4 dashboard.html — Customer dashboard with balance & live statement

2.5 transfer.html — Inter-account fund transfer screen

2.6 style.css — Shared responsive CSS stylesheet

2.7 app.js — Frontend REST API integration & session state

PART 3: CONFIGURATION, PERSISTENCE & DOCUMENTATION

3.1 run-server.bat — Windows one-click batch launcher

3.2 accounts.txt — Comma-separated account database

3.3 README.md — Full documentation & compilation guide

1.1 Account.java

Path: `BankJavaApp/Account.java`

Java - Entity Class representing bank account details

```java
/**
 * Account.java
 *
 * Purpose:
 * Represents a single bank account in the Bank Management System.
 * Stores account credentials (account number, holder name, PIN) and maintains balance.
 * Provides standard getters, setters, and helper methods for updating balance.
 *
 * Target Audience: 2nd-Year Engineering Students learning Core Java & OOP Concepts.
 */

public class Account {
    // Attributes (Encapsulation: private fields with public getters/setters)
    private String accountNumber;
    private String holderName;
    private String pin;
    private double balance;

    /**
     * Parameterized Constructor to initialize a new Account object.
     *
     * @param accountNumber Unique account identifier (e.g. "1001")
     * @param holderName    Full name of the account holder
     * @param pin           4-digit secret PIN code
     * @param balance       Initial opening balance
     */
    public Account(String accountNumber, String holderName, String pin, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.pin = pin;
        this.balance = balance;
    }

    // ==================== GETTERS AND SETTERS ====================

    public String getAccountNumber() {
        return accountNumber;
    }

    public void setAccountNumber(String accountNumber) {
        this.accountNumber = accountNumber;
    }

    public String getHolderName() {
        return holderName;
    }

    public void setHolderName(String holderName) {
        this.holderName = holderName;
    }

    public String getPin() {
        return pin;
    }

    public void setPin(String pin) {
        this.pin = pin;
    }

    public double getBalance() {
```

```
        return balance;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    // ================== BUSINESS METHODS ==================

    /**
     * Adds an amount to the current balance.
     *
     * @param amount Positive monetary value to deposit
     */
    public void credit(double amount) {
        if (amount > 0) {
            this.balance += amount;
        }
    }

    /**
     * Deducts an amount from the current balance if funds are sufficient.
     *
     * @param amount Positive monetary value to withdraw
     * @return true if deduction succeeded, false otherwise
     */
    public boolean debit(double amount) {
        if (amount > 0 && this.balance >= amount) {
            this.balance -= amount;
            return true;
        }
        return false;
    }

    /**
     * Validates if the provided PIN matches this account's PIN.
     *
     * @param enteredPin PIN entered by the user
     * @return true if valid, false otherwise
     */
    public boolean validatePin(String enteredPin) {
        return this.pin != null && this.pin.equals(enteredPin);
    }

    /**
     * String representation of an Account for display purposes.
     */
    @Override
    public String toString() {
        return String.format(
            "---------------------------------------\n" +
            " Account Number : %s\n" +
            " Holder Name    : %s\n" +
            " Current Balance: $%.2f\n" +
            "---------------------------------------",
            accountNumber, holderName, balance
        );
    }
}
```

1.2 Transaction.java

Path: `BankJavaApp/Transaction.java`

Java - Entity Class representing financial transactions

```java
/**
 * Transaction.java
 *
 * Purpose:
 * Represents a single financial transaction (Deposit, Withdrawal, or Transfer).
 * Records the transaction type, monetary amount, timestamp, related account number,
 * and optional remarks (e.g. transfer destination).
 *
 * Target Audience: 2nd-Year Engineering Students learning Core Java & OOP Concepts.
 */

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Transaction {
    // Transaction details
    private String transactionId;
    private String type;           // "Deposit", "Withdraw", or "Transfer"
    private double amount;
    private String date;           // Formatted date string
    private String accountNumber;  // Account associated with this transaction
    private String remarks;        // Additional note (e.g., "To Acc: 1002")

    // Formatter for readable date-time presentation
    private static final DateTimeFormatter FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    /**
     * Parameterized Constructor when automatic timestamping is desired.
     *
     * @param type          "Deposit", "Withdraw", or "Transfer"
     * @param amount        Transaction monetary value
     * @param accountNumber Account number associated
     * @param remarks       Additional context / remarks
     */
    public Transaction(String type, double amount, String accountNumber, String remarks) {
        this.transactionId = "TXN" + System.currentTimeMillis() % 100000;
        this.type = type;
        this.amount = amount;
        this.date = LocalDateTime.now().format(FORMATTER);
        this.accountNumber = accountNumber;
        this.remarks = remarks;
    }

    /**
     * Parameterized Constructor when date is explicitly provided (e.g. if loaded from storage).
     */
    public Transaction(String transactionId, String type, double amount, String date, String accountNumber, String remarks) {
        this.transactionId = transactionId;
        this.type = type;
        this.amount = amount;
        this.date = date;
        this.accountNumber = accountNumber;
        this.remarks = remarks;
    }

    // ==================== GETTERS AND SETTERS ====================

    public String getTransactionId() {
        return transactionId;
    }
```

```
    public String getType() {
        return type;
    }

    public double getAmount() {
        return amount;
    }

    public String getDate() {
        return date;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getRemarks() {
        return remarks;
    }

    /**
     * Formats transaction as a single structured table row for mini statements.
     */
    @Override
    public String toString() {
        return String.format("%-10s | %-19s | %-10s | $%-9.2f | %s",
            transactionId, date, type, amount, remarks);
    }
}
```

1.3 Bank.java

Path: `BankJavaApp/Bank.java`

Java - Core Business Logic Engine & Validations

```java
/**
 * Bank.java
 *
 * Purpose:
 * Core business logic engine for the Bank Management System.
 * Manages the collection of active accounts and transaction logs.
 * Enforces business rules and validations (e.g. balance checks, PIN authentication,
 * duplicate account detection, fund transfer rules, and mini statement generation).
 *
 * Target Audience: 2nd-Year Engineering Students learning Core Java & Business Logic.
 */

import java.util.ArrayList;
import java.util.List;

public class Bank {
    // In-memory data structures using Java Collections
    private ArrayList<Account> accounts;
    private ArrayList<Transaction> transactions;

    /**
     * Default Constructor.
     */
    public Bank() {
        this.accounts = new ArrayList<>();
        this.transactions = new ArrayList<>();
    }

    /**
     * Constructor initialized with preloaded accounts.
     *
     * @param initialAccounts List of accounts loaded from storage
     */
    public Bank(ArrayList<Account> initialAccounts) {
        this.accounts = (initialAccounts != null) ? initialAccounts : new ArrayList<>();
        this.transactions = new ArrayList<>();
    }

    // ================== GETTERS ==================

    public ArrayList<Account> getAccounts() {
        return accounts;
    }

    public ArrayList<Transaction> getTransactions() {
        return transactions;
    }

    // ================== ACCOUNT OPERATIONS ==================

    /**
     * Searches for an account by its unique account number.
     *
     * @param accountNumber Account number to search for
     * @return Account object if found, or null if not found
     */
    public Account findAccount(String accountNumber) {
        if (accountNumber == null || accountNumber.trim().isEmpty()) {
            return null;
        }
```

```
        for (Account acc : accounts) {
            if (acc.getAccountNumber().equalsIgnoreCase(accountNumber.trim())) {
                return acc;
            }
        }
        return null;
    }

    /**
     * Creates and registers a new account after validating inputs.
     *
     * @param accountNumber  Desired unique account number
     * @param holderName     Name of the account holder
     * @param pin            4-digit security PIN
     * @param initialDeposit Opening balance (must be non-negative)
     * @return true if created successfully, false otherwise
     */
    public boolean createAccount(String accountNumber, String holderName, String pin, double initialDeposit) {
        // Validation: Blank inputs
        if (accountNumber == null || accountNumber.trim().isEmpty() ||
            holderName == null || holderName.trim().isEmpty() ||
            pin == null || pin.trim().isEmpty()) {
            System.out.println("[Error] Account number, holder name, and PIN cannot be empty.");
            return false;
        }

        // Validation: Unique account number
        if (findAccount(accountNumber) != null) {
            System.out.println("[Error] Account number '" + accountNumber + "' already exists! Please choose another.");
            return false;
        }

        // Validation: Non-negative initial deposit
        if (initialDeposit < 0) {
            System.out.println("[Error] Initial deposit cannot be negative.");
            return false;
        }

        // Create and add account
        Account newAcc = new Account(accountNumber.trim(), holderName.trim(), pin.trim(), initialDeposit);
        accounts.add(newAcc);

        // Record initial deposit transaction if amount > 0
        if (initialDeposit > 0) {
            transactions.add(new Transaction("Deposit", initialDeposit, newAcc.getAccountNumber(), "Initial Opening Deposit"));
        }

        System.out.println("\n[Success] Account successfully created for " + holderName + "!");
        System.out.println(newAcc);
        return true;
    }

    /**
     * Authenticates account credentials.
     *
     * @param accountNumber Account number
     * @param pin           Security PIN
     * @return Account object if login is successful, null otherwise
     */
    public Account login(String accountNumber, String pin) {
        Account acc = findAccount(accountNumber);
        if (acc == null) {
            System.out.println("[Error] Account number '" + accountNumber + "' not found.");
            return null;
        }

        if (!acc.validatePin(pin)) {
            System.out.println("[Error] Invalid PIN entered for account: " + accountNumber);
```

```
            return null;
        }

        System.out.println("[Success] Login successful. Welcome back, " + acc.getHolderName() + "!");
        return acc;
    }

    /**
     * Deposits money into a specified account.
     *
     * @param accountNumber Target account number
     * @param amount        Amount to deposit (must be > 0)
     * @return true if deposit succeeds, false otherwise
     */
    public boolean deposit(String accountNumber, double amount) {
        if (amount <= 0) {
            System.out.println("[Error] Deposit amount must be greater than zero.");
            return false;
        }

        Account acc = findAccount(accountNumber);
        if (acc == null) {
            System.out.println("[Error] Account not found: " + accountNumber);
            return false;
        }

        acc.credit(amount);
        transactions.add(new Transaction("Deposit", amount, acc.getAccountNumber(), "Cash / Online Deposit"));

        System.out.printf("\n[Success] Successfully deposited $%.2f into Account %s.\n", amount, acc.getAccountNumber());
        System.out.printf("Updated Balance: $%.2f\n", acc.getBalance());
        return true;
    }

    /**
     * Withdraws money from an account after validating PIN and sufficient funds.
     *
     * @param accountNumber Account number
     * @param pin           Security PIN
     * @param amount        Amount to withdraw (must be > 0 and <= balance)
     * @return true if withdrawal succeeds, false otherwise
     */
    public boolean withdraw(String accountNumber, String pin, double amount) {
        if (amount <= 0) {
            System.out.println("[Error] Withdrawal amount must be greater than zero.");
            return false;
        }

        Account acc = findAccount(accountNumber);
        if (acc == null) {
            System.out.println("[Error] Account not found: " + accountNumber);
            return false;
        }

        if (!acc.validatePin(pin)) {
            System.out.println("[Error] Incorrect PIN! Withdrawal denied.");
            return false;
        }

        if (acc.getBalance() < amount) {
            System.out.printf("[Error] Insufficient funds! Available balance: $%.2f, Requested: $%.2f\n",
                acc.getBalance(), amount);
            return false;
        }

        acc.debit(amount);
        transactions.add(new Transaction("Withdraw", amount, acc.getAccountNumber(), "ATM / Branch Withdrawal"));
```

```
        System.out.printf("\n[Success] Successfully withdrew $%.2f from Account %s.\n", amount, acc.getAccountNumber());
        System.out.printf("Remaining Balance: $%.2f\n", acc.getBalance());
        return true;
    }

    /**
     * Transfers money between two accounts with security verification.
     *
     * @param fromAccNo Source account number
     * @param fromPin   Source account PIN
     * @param toAccNo   Destination account number
     * @param amount    Amount to transfer (must be > 0 and <= sender balance)
     * @return true if transfer succeeds, false otherwise
     */
    public boolean transfer(String fromAccNo, String fromPin, String toAccNo, double amount) {
        if (amount <= 0) {
            System.out.println("[Error] Transfer amount must be greater than zero.");
            return false;
        }

        if (fromAccNo.equalsIgnoreCase(toAccNo)) {
            System.out.println("[Error] Sender and receiver account numbers cannot be the same!");
            return false;
        }

        Account sender = findAccount(fromAccNo);
        if (sender == null) {
            System.out.println("[Error] Source account not found: " + fromAccNo);
            return false;
        }

        if (!sender.validatePin(fromPin)) {
            System.out.println("[Error] Incorrect sender PIN! Transfer aborted.");
            return false;
        }

        Account receiver = findAccount(toAccNo);
        if (receiver == null) {
            System.out.println("[Error] Beneficiary account not found: " + toAccNo);
            return false;
        }

        if (sender.getBalance() < amount) {
            System.out.printf("[Error] Insufficient balance for transfer! Available: $%.2f, Requested: $%.2f\n",
                sender.getBalance(), amount);
            return false;
        }

        // Execute transfer
        sender.debit(amount);
        receiver.credit(amount);

        // Record logs for both parties
        transactions.add(new Transaction("Transfer", amount, sender.getAccountNumber(), "Transferred to Acc: " +
receiver.getAccountNumber()));
        transactions.add(new Transaction("Deposit", amount, receiver.getAccountNumber(), "Received from Acc: " +
sender.getAccountNumber()));

        System.out.printf("\n[Success] Successfully transferred $%.2f to %s (Acc: %s)!\n",
            amount, receiver.getHolderName(), receiver.getAccountNumber());
        System.out.printf("Sender's Remaining Balance: $%.2f\n", sender.getBalance());
        return true;
    }

    /**
     * Displays mini statement of transactions for a specific account.
     *
     * @param accountNumber Account number to generate statement for
```

```
     */
    public void printMiniStatement(String accountNumber) {
        Account acc = findAccount(accountNumber);
        if (acc == null) {
            System.out.println("[Error] Account not found: " + accountNumber);
            return;
        }

        System.out.println("\n================================================================================");
        System.out.println("                              MINI STATEMENT                                    ");
        System.out.println("================================================================================");
        System.out.println("Account Number : " + acc.getAccountNumber());
        System.out.println("Account Holder : " + acc.getHolderName());
        System.out.printf("Current Balance: $%.2f\n", acc.getBalance());
        System.out.println("--------------------------------------------------------------------------------");
        System.out.printf("%-10s | %-19s | %-10s | %-10s | %s\n", "TXN ID", "DATE & TIME", "TYPE", "AMOUNT", "DETAILS");
        System.out.println("--------------------------------------------------------------------------------");

        int count = 0;
        for (Transaction txn : transactions) {
            if (txn.getAccountNumber().equalsIgnoreCase(acc.getAccountNumber())) {
                System.out.println(txn);
                count++;
            }
        }

        if (count == 0) {
            System.out.println("No transaction history recorded for this session.");
        }
        System.out.println("================================================================================\n");
    }

    /**
     * Retrieves the list of transactions associated with a specific account number.
     *
     * @param accountNumber Target account number
     * @return List of Transaction objects
     */
    public List<Transaction> getTransactionsForAccount(String accountNumber) {
        List<Transaction> list = new ArrayList<>();
        if (accountNumber == null) return list;
        for (Transaction txn : transactions) {
            if (txn.getAccountNumber().equalsIgnoreCase(accountNumber.trim())) {
                list.add(txn);
            }
        }
        return list;
    }
}
```

1.4 FileHandler.java

Path: `BankJavaApp/FileHandler.java`

Java - Persistence Layer reading/writing accounts.txt

```java
/**
 * FileHandler.java
 *
 * Purpose:
 * Provides file persistence capabilities for the Bank Management System.
 * Reads and writes account records to a simple comma-separated text file (accounts.txt).
 * This ensures customer accounts persist even after closing and restarting the application.
 *
 * Target Audience: 2nd-Year Engineering Students learning File I/O in Java.
 */

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class FileHandler {
    // Default file name used for saving accounts
    private static final String DEFAULT_FILE_NAME = "accounts.txt";

    /**
     * Saves all accounts to a comma-separated text file.
     * Format per line: accountNumber,holderName,pin,balance
     *
     * @param accounts List of Account objects to persist
     * @param filePath Path or filename where data should be saved
     * @return true if save was successful, false otherwise
     */
    public static boolean saveAccounts(List<Account> accounts, String filePath) {
        String path = (filePath != null && !filePath.trim().isEmpty()) ? filePath : DEFAULT_FILE_NAME;

        try (BufferedWriter writer = new BufferedWriter(new FileWriter(path))) {
            for (Account acc : accounts) {
                // Write line: 1001,John Doe,1234,1500.00
                String line = String.format("%s,%s,%s,%.2f",
                    acc.getAccountNumber(),
                    acc.getHolderName(),
                    acc.getPin(),
                    acc.getBalance()
                );
                writer.write(line);
                writer.newLine();
            }
            return true;
        } catch (IOException e) {
            System.err.println("[FileHandler Error] Failed to save accounts to file: " + e.getMessage());
            return false;
        }
    }

    /**
     * Saves accounts using default file path ("accounts.txt").
     */
    public static boolean saveAccounts(List<Account> accounts) {
        return saveAccounts(accounts, DEFAULT_FILE_NAME);
    }
```

```
    /**
     * Reads accounts from a comma-separated text file on startup.
     * If the file does not exist yet, returns an empty list without throwing an error.
     *
     * @param filePath Path or filename to load from
     * @return List of parsed Account objects
     */
    public static ArrayList<Account> loadAccounts(String filePath) {
        ArrayList<Account> accounts = new ArrayList<>();
        String path = (filePath != null && !filePath.trim().isEmpty()) ? filePath : DEFAULT_FILE_NAME;
        File file = new File(path);

        // If file doesn't exist yet (first run), return empty list
        if (!file.exists()) {
            System.out.println("[FileHandler Info] '" + path + "' not found. Initializing with empty account store.");
            return accounts;
        }

        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            String line;
            int lineNumber = 0;

            while ((line = reader.readLine()) != null) {
                lineNumber++;
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) {
                    continue; // Skip blank lines and comments
                }

                // Split by comma
                String[] parts = line.split(",");
                if (parts.length == 4) {
                    try {
                        String accNo = parts[0].trim();
                        String name = parts[1].trim();
                        String pin = parts[2].trim();
                        double balance = Double.parseDouble(parts[3].trim());

                        accounts.add(new Account(accNo, name, pin, balance));
                    } catch (NumberFormatException e) {
                        System.err.println("[FileHandler Warning] Skipping corrupted line " + lineNumber + ": " + line);
                    }
                } else {
                    System.err.println("[FileHandler Warning] Invalid data format on line " + lineNumber + ": " + line);
                }
            }
            System.out.println("[FileHandler Info] Successfully loaded " + accounts.size() + " account(s) from " + path);
        } catch (IOException e) {
            System.err.println("[FileHandler Error] Error reading accounts file: " + e.getMessage());
        }

        return accounts;
    }

    /**
     * Loads accounts using default file path ("accounts.txt").
     */
    public static ArrayList<Account> loadAccounts() {
        return loadAccounts(DEFAULT_FILE_NAME);
    }
}
```

1.5 Main.java

Path: `BankJavaApp/Main.java`

Java - Interactive Console CLI Application

```java
/**
 * Main.java
 *
 * Purpose:
 * Entry point and console user interface (CLI) for the Bank Management System.
 * Presents a numbered menu, reads input from user via Scanner, handles exceptions,
 * maintains session state (logged-in user), and coordinates with Bank and FileHandler.
 *
 * Target Audience: 2nd-Year Engineering Students learning Console I/O and Control Flow.
 */

import java.util.Scanner;

public class Main {
    private static Bank bank;
    private static Scanner scanner;
    private static Account currentSession = null; // Currently logged in user (optional convenience)
    private static final String DATA_FILE = "accounts.txt";

    public static void main(String[] args) {
        scanner = new Scanner(System.in);

        // 1. Startup: Load existing accounts from accounts.txt
        System.out.println("================================================");
        System.out.println("       WELCOME TO THE BANK MANAGEMENT SYSTEM     ");
        System.out.println("================================================");
        bank = new Bank(FileHandler.loadAccounts(DATA_FILE));

        boolean running = true;

        // 2. Main Application Loop
        while (running) {
            printMenu();
            System.out.print("Enter your choice (1-8): ");
            String input = scanner.nextLine().trim();

            switch (input) {
                case "1":
                    handleCreateAccount();
                    break;
                case "2":
                    handleLogin();
                    break;
                case "3":
                    handleDeposit();
                    break;
                case "4":
                    handleWithdraw();
                    break;
                case "5":
                    handleTransfer();
                    break;
                case "6":
                    handleCheckBalance();
                    break;
                case "7":
                    handleMiniStatement();
                    break;
                case "8":
                    // 3. Exit: Persist accounts to file and terminate
```

```
                handleExit();
                running = false;
                break;
            default:
                System.out.println("[Error] Invalid choice! Please enter a number between 1 and 8.\n");
        }
    }

    scanner.close();
}

/**
 * Prints the primary console navigation menu.
 */
private static void printMenu() {
    System.out.println("\n-------------------------------------------------");
    System.out.println("                   MAIN MENU                     ");
    if (currentSession != null) {
        System.out.println(" Active User: " + currentSession.getHolderName() +
                           " (Acc: " + currentSession.getAccountNumber() + ")");
    } else {
        System.out.println(" Active User: [None - Logged Out]");
    }
    System.out.println("-------------------------------------------------");
    System.out.println(" 1. Create New Account");
    System.out.println(" 2. Login to Account");
    System.out.println(" 3. Deposit Money");
    System.out.println(" 4. Withdraw Money");
    System.out.println(" 5. Transfer Funds");
    System.out.println(" 6. Check Account Balance");
    System.out.println(" 7. Mini Statement (Transaction History)");
    System.out.println(" 8. Exit Application");
    System.out.println("-------------------------------------------------");
}

/**
 * Handler for Option 1: Create Account
 */
private static void handleCreateAccount() {
    System.out.println("\n--- [ CREATE NEW ACCOUNT ] ---");
    try {
        System.out.print("Enter Desired Account Number (e.g. 1001): ");
        String accNo = scanner.nextLine().trim();

        System.out.print("Enter Account Holder Full Name: ");
        String name = scanner.nextLine().trim();

        System.out.print("Set a 4-digit Security PIN: ");
        String pin = scanner.nextLine().trim();

        System.out.print("Enter Initial Deposit Amount ($): ");
        double initialDeposit = readDoubleInput();

        boolean success = bank.createAccount(accNo, name, pin, initialDeposit);
        if (success) {
            // Auto-save on account creation
            FileHandler.saveAccounts(bank.getAccounts(), DATA_FILE);
        }
    } catch (Exception e) {
        System.out.println("[Error] Failed to create account due to unexpected input: " + e.getMessage());
    }
}

/**
 * Handler for Option 2: Login
 */
private static void handleLogin() {
    System.out.println("\n--- [ ACCOUNT LOGIN ] ---");
```

```
        System.out.print("Enter Account Number: ");
        String accNo = scanner.nextLine().trim();

        System.out.print("Enter PIN: ");
        String pin = scanner.nextLine().trim();

        Account acc = bank.login(accNo, pin);
        if (acc != null) {
            currentSession = acc;
            System.out.println("You are now actively logged in as " + acc.getHolderName() + ".");
        }
    }

    /**
     * Handler for Option 3: Deposit
     */
    private static void handleDeposit() {
        System.out.println("\n--- [ DEPOSIT MONEY ] ---");
        String accNo;

        if (currentSession != null) {
            System.out.println("Using Active Account: " + currentSession.getAccountNumber());
            accNo = currentSession.getAccountNumber();
        } else {
            System.out.print("Enter Target Account Number: ");
            accNo = scanner.nextLine().trim();
        }

        System.out.print("Enter Amount to Deposit ($): ");
        double amount = readDoubleInput();

        boolean success = bank.deposit(accNo, amount);
        if (success) {
            FileHandler.saveAccounts(bank.getAccounts(), DATA_FILE);
        }
    }

    /**
     * Handler for Option 4: Withdraw
     */
    private static void handleWithdraw() {
        System.out.println("\n--- [ WITHDRAW MONEY ] ---");
        String accNo;

        if (currentSession != null) {
            accNo = currentSession.getAccountNumber();
            System.out.println("Using Active Account: " + accNo);
        } else {
            System.out.print("Enter Account Number: ");
            accNo = scanner.nextLine().trim();
        }

        System.out.print("Enter Security PIN: ");
        String pin = scanner.nextLine().trim();

        System.out.print("Enter Amount to Withdraw ($): ");
        double amount = readDoubleInput();

        boolean success = bank.withdraw(accNo, pin, amount);
        if (success) {
            FileHandler.saveAccounts(bank.getAccounts(), DATA_FILE);
        }
    }

    /**
     * Handler for Option 5: Transfer
     */
    private static void handleTransfer() {
```

```
        System.out.println("\n--- [ TRANSFER FUNDS ] ---");
        String fromAccNo;

        if (currentSession != null) {
            fromAccNo = currentSession.getAccountNumber();
            System.out.println("Sender Account (Active): " + fromAccNo);
        } else {
            System.out.print("Enter Your (Sender) Account Number: ");
            fromAccNo = scanner.nextLine().trim();
        }

        System.out.print("Enter Sender PIN: ");
        String pin = scanner.nextLine().trim();

        System.out.print("Enter Beneficiary (Receiver) Account Number: ");
        String toAccNo = scanner.nextLine().trim();

        System.out.print("Enter Amount to Transfer ($): ");
        double amount = readDoubleInput();

        boolean success = bank.transfer(fromAccNo, pin, toAccNo, amount);
        if (success) {
            FileHandler.saveAccounts(bank.getAccounts(), DATA_FILE);
        }
    }

    /**
     * Handler for Option 6: Check Balance
     */
    private static void handleCheckBalance() {
        System.out.println("\n--- [ CHECK BALANCE ] ---");
        String accNo;

        if (currentSession != null) {
            accNo = currentSession.getAccountNumber();
        } else {
            System.out.print("Enter Account Number: ");
            accNo = scanner.nextLine().trim();
        }

        System.out.print("Enter Security PIN: ");
        String pin = scanner.nextLine().trim();

        Account acc = bank.findAccount(accNo);
        if (acc == null) {
            System.out.println("[Error] Account number not found: " + accNo);
            return;
        }

        if (!acc.validatePin(pin)) {
            System.out.println("[Error] Incorrect PIN! Access denied.");
            return;
        }

        System.out.println("\n=======================================");
        System.out.println(" Account Number: " + acc.getAccountNumber());
        System.out.println(" Holder Name   : " + acc.getHolderName());
        System.out.printf(" Current Balance: $%.2f\n", acc.getBalance());
        System.out.println("=======================================");
    }

    /**
     * Handler for Option 7: Mini Statement
     */
    private static void handleMiniStatement() {
        System.out.println("\n--- [ MINI STATEMENT ] ---");
        String accNo;
```

```
        if (currentSession != null) {
            accNo = currentSession.getAccountNumber();
        } else {
            System.out.print("Enter Account Number: ");
            accNo = scanner.nextLine().trim();
        }

        System.out.print("Enter Security PIN: ");
        String pin = scanner.nextLine().trim();

        Account acc = bank.findAccount(accNo);
        if (acc == null) {
            System.out.println("[Error] Account not found: " + accNo);
            return;
        }

        if (!acc.validatePin(pin)) {
            System.out.println("[Error] Incorrect PIN! Access denied.");
            return;
        }

        bank.printMiniStatement(accNo);
    }

    /**
     * Handler for Option 8: Exit
     */
    private static void handleExit() {
        System.out.println("\nSaving account data to " + DATA_FILE + "...");
        FileHandler.saveAccounts(bank.getAccounts(), DATA_FILE);
        System.out.println("================================================");
        System.out.println(" Thank you for banking with us! Have a great day.");
        System.out.println("================================================");
    }

    /**
     * Helper to reliably read double numeric values without Scanner buffer issues.
     */
    private static double readDoubleInput() {
        while (true) {
            try {
                String input = scanner.nextLine().trim();
                return Double.parseDouble(input);
            } catch (NumberFormatException e) {
                System.out.print("[Error] Invalid numeric amount. Please re-enter: ");
            }
        }
    }
}
```

1.6 BankWebServer.java

Path: `BankJavaApp/BankWebServer.java`

Java - Built-in HTTP Web Server & REST API Endpoints

```
/**
 * BankWebServer.java
 *
 * Purpose:
 * Lightweight built-in HTTP Web Server for the Bank Management System.
 * Uses core Java standard library (com.sun.net.httpserver.HttpServer) with zero external frameworks.
 * Serves the static HTML/CSS/JS frontend on port 8080 and provides REST APIs for
 * Account Registration, Login, Deposit, Withdrawal, Transfer, and Statement Generation.
 *
 * Target Audience: 2nd-Year Engineering Students learning Full-Stack Web Development in Core Java.
 */

import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpExchange;

import java.io.*;
import java.net.InetSocketAddress;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.*;
import java.util.concurrent.Executors;

public class BankWebServer {
    private static final int PORT = 8080;
    private static final String DATA_FILE = "accounts.txt";
    private static Bank bank;
    private static File webUiDir;

    public static void main(String[] args) {
        try {
            // 1. Initialize data store
            System.out.println("================================================");
            System.out.println("     STARTING APEX BANK HTTP WEB SERVER         ");
            System.out.println("================================================");

            ArrayList<Account> loadedAccounts = FileHandler.loadAccounts(DATA_FILE);
            bank = new Bank(loadedAccounts);

            // 2. Locate the Web UI Directory (BankWebUI)
            locateWebUiDirectory();

            // 3. Create HTTP Server on port 8080
            HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
            server.setExecutor(Executors.newCachedThreadPool());

            // 4. Register REST API Handlers
            server.createContext("/api/register", new RegisterHandler());
            server.createContext("/api/login", new LoginHandler());
            server.createContext("/api/account", new AccountDetailsHandler());
            server.createContext("/api/deposit", new DepositHandler());
            server.createContext("/api/withdraw", new WithdrawHandler());
            server.createContext("/api/transfer", new TransferHandler());
            server.createContext("/api/statement", new StatementHandler());

            // 5. Register Static File Handler (Root)
            server.createContext("/", new StaticFileHandler());

            // 6. Start the server
```

```
            server.start();
            System.out.println("\n🚀 Bank Web Server is running successfully!");
            System.out.println("🌐 Open your browser and visit: http://localhost:" + PORT);
            System.out.println("📂 Serving Web UI from: " + webUiDir.getAbsolutePath());
            System.out.println("🛑 Press Ctrl + C in this terminal to stop the server.\n");

        } catch (IOException e) {
            System.err.println("[Server Error] Failed to start server: " + e.getMessage());
            e.printStackTrace();
        }
    }

    /**
     * Finds the BankWebUI folder whether executed from BankJavaApp or project root.
     */
    private static void locateWebUiDirectory() {
        File[] candidatePaths = new File[] {
            new File("BankWebUI"),
            new File("../BankWebUI"),
            new File("BankManagementSystem/BankWebUI"),
            new File("../../BankWebUI")
        };

        for (File path : candidatePaths) {
            if (path.exists() && path.isDirectory()) {
                webUiDir = path;
                return;
            }
        }
        // Default fallback
        webUiDir = new File("BankWebUI");
    }

    // ==================== REST API HANDLERS ====================

    /**
     * Handler: POST /api/register
     */
    static class RegisterHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            addCorsHeaders(exchange);
            if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(204, -1);
                return;
            }

            if (!"POST".equalsIgnoreCase(exchange.getRequestMethod())) {
                sendJsonResponse(exchange, 405, jsonResponse(false, "Method not allowed. Use POST."));
                return;
            }

            Map<String, String> params = parseRequestBody(exchange);
            String accNo = params.get("accountNumber");
            String name = params.get("holderName");
            String pin = params.get("pin");
            double initialDeposit = parseDouble(params.get("initialDeposit"), 0.0);

            if (accNo == null || accNo.trim().isEmpty() || name == null || name.trim().isEmpty() || pin == null ||
pin.trim().isEmpty()) {
                sendJsonResponse(exchange, 400, jsonResponse(false, "Account number, name, and PIN are required."));
                return;
            }

            if (bank.findAccount(accNo) != null) {
                sendJsonResponse(exchange, 409, jsonResponse(false, "Account number '" + accNo + "' is already registered!"));
                return;
            }
```

```
            boolean created = bank.createAccount(accNo, name, pin, initialDeposit);
            if (created) {
                FileHandler.saveAccounts(bank.getAccounts(), DATA_FILE);
                sendJsonResponse(exchange, 200, "{\"success\":true,\"message\":\"Account created
successfully!\",\"accountNumber\":\"" + escapeJson(accNo) + "\",\"holderName\":\"" + escapeJson(name) + "\",\"balance\":" +
initialDeposit + "}");
            } else {
                sendJsonResponse(exchange, 400, jsonResponse(false, "Could not create account. Check inputs."));
            }
        }
    }

    /**
     * Handler: POST /api/login
     */
    static class LoginHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            addCorsHeaders(exchange);
            if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(204, -1);
                return;
            }

            if (!"POST".equalsIgnoreCase(exchange.getRequestMethod())) {
                sendJsonResponse(exchange, 405, jsonResponse(false, "Method not allowed. Use POST."));
                return;
            }

            Map<String, String> params = parseRequestBody(exchange);
            String accNo = params.get("accountNumber");
            String pin = params.get("pin");

            Account acc = bank.findAccount(accNo);
            if (acc == null) {
                sendJsonResponse(exchange, 404, jsonResponse(false, "Account number '" + accNo + "' not found."));
                return;
            }

            if (!acc.validatePin(pin)) {
                sendJsonResponse(exchange, 401, jsonResponse(false, "Invalid security PIN entered."));
                return;
            }

            String response = String.format(
                "{\"success\":true,\"message\":\"Login
successful!\",\"accountNumber\":\"%s\",\"holderName\":\"%s\",\"balance\":%.2f}",
                escapeJson(acc.getAccountNumber()), escapeJson(acc.getHolderName()), acc.getBalance()
            );
            sendJsonResponse(exchange, 200, response);
        }
    }

    /**
     * Handler: GET /api/account?accountNumber=...&pin=...
     */
    static class AccountDetailsHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            addCorsHeaders(exchange);
            if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(204, -1);
                return;
            }

            Map<String, String> queryParams = parseQueryParams(exchange.getRequestURI().getQuery());
            String accNo = queryParams.get("accountNumber");
```

```
            String pin = queryParams.get("pin");

            Account acc = bank.findAccount(accNo);
            if (acc == null) {
                sendJsonResponse(exchange, 404, jsonResponse(false, "Account not found."));
                return;
            }

            if (pin != null && !acc.validatePin(pin)) {
                sendJsonResponse(exchange, 401, jsonResponse(false, "Incorrect PIN."));
                return;
            }

            String response = String.format(
                "{\"success\":true,\"accountNumber\":\"%s\",\"holderName\":\"%s\",\"balance\":%.2f}",
                escapeJson(acc.getAccountNumber()), escapeJson(acc.getHolderName()), acc.getBalance()
            );
            sendJsonResponse(exchange, 200, response);
        }
    }

    /**
     * Handler: POST /api/deposit
     */
    static class DepositHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            addCorsHeaders(exchange);
            if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(204, -1);
                return;
            }

            Map<String, String> params = parseRequestBody(exchange);
            String accNo = params.get("accountNumber");
            double amount = parseDouble(params.get("amount"), 0.0);

            if (amount <= 0) {
                sendJsonResponse(exchange, 400, jsonResponse(false, "Deposit amount must be greater than zero."));
                return;
            }

            Account acc = bank.findAccount(accNo);
            if (acc == null) {
                sendJsonResponse(exchange, 404, jsonResponse(false, "Account not found."));
                return;
            }

            boolean success = bank.deposit(accNo, amount);
            if (success) {
                FileHandler.saveAccounts(bank.getAccounts(), DATA_FILE);
                sendJsonResponse(exchange, 200, String.format(
                    "{\"success\":true,\"message\":\"Successfully deposited $%.2f\",\"balance\":%.2f}",
                    amount, acc.getBalance()
                ));
            } else {
                sendJsonResponse(exchange, 400, jsonResponse(false, "Deposit failed."));
            }
        }
    }

    /**
     * Handler: POST /api/withdraw
     */
    static class WithdrawHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            addCorsHeaders(exchange);
```

```
            if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(204, -1);
                return;
            }

            Map<String, String> params = parseRequestBody(exchange);
            String accNo = params.get("accountNumber");
            String pin = params.get("pin");
            double amount = parseDouble(params.get("amount"), 0.0);

            Account acc = bank.findAccount(accNo);
            if (acc == null) {
                sendJsonResponse(exchange, 404, jsonResponse(false, "Account not found."));
                return;
            }

            if (!acc.validatePin(pin)) {
                sendJsonResponse(exchange, 401, jsonResponse(false, "Incorrect PIN. Withdrawal denied."));
                return;
            }

            if (amount <= 0) {
                sendJsonResponse(exchange, 400, jsonResponse(false, "Withdrawal amount must be greater than zero."));
                return;
            }

            if (acc.getBalance() < amount) {
                sendJsonResponse(exchange, 400, jsonResponse(false, String.format("Insufficient funds! Balance: $%.2f",
acc.getBalance())));
                return;
            }

            boolean success = bank.withdraw(accNo, pin, amount);
            if (success) {
                FileHandler.saveAccounts(bank.getAccounts(), DATA_FILE);
                sendJsonResponse(exchange, 200, String.format(
                    "{\"success\":true,\"message\":\"Successfully withdrew $%.2f\",\"balance\":%.2f}",
                    amount, acc.getBalance()
                ));
            } else {
                sendJsonResponse(exchange, 400, jsonResponse(false, "Withdrawal failed."));
            }
        }
    }

    /**
     * Handler: POST /api/transfer
     */
    static class TransferHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            addCorsHeaders(exchange);
            if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(204, -1);
                return;
            }

            Map<String, String> params = parseRequestBody(exchange);
            String fromAcc = params.get("fromAccount");
            String pin = params.get("pin");
            String toAcc = params.get("toAccount");
            double amount = parseDouble(params.get("amount"), 0.0);

            if (fromAcc == null || toAcc == null || fromAcc.equalsIgnoreCase(toAcc)) {
                sendJsonResponse(exchange, 400, jsonResponse(false, "Source and destination accounts must be different."));
                return;
            }
```

```
            Account sender = bank.findAccount(fromAcc);
            if (sender == null) {
                sendJsonResponse(exchange, 404, jsonResponse(false, "Sender account not found."));
                return;
            }

            if (!sender.validatePin(pin)) {
                sendJsonResponse(exchange, 401, jsonResponse(false, "Incorrect sender PIN!"));
                return;
            }

            Account recipient = bank.findAccount(toAcc);
            if (recipient == null) {
                sendJsonResponse(exchange, 404, jsonResponse(false, "Beneficiary account '" + toAcc + "' not found."));
                return;
            }

            if (amount <= 0) {
                sendJsonResponse(exchange, 400, jsonResponse(false, "Transfer amount must be positive."));
                return;
            }

            if (sender.getBalance() < amount) {
                sendJsonResponse(exchange, 400, jsonResponse(false, String.format("Insufficient funds! Available: $%.2f",
sender.getBalance())));
                return;
            }

            boolean success = bank.transfer(fromAcc, pin, toAcc, amount);
            if (success) {
                FileHandler.saveAccounts(bank.getAccounts(), DATA_FILE);
                sendJsonResponse(exchange, 200, String.format(
                    "{\"success\":true,\"message\":\"Transferred $%.2f to %s (Acc: %s) successfully!\",\"balance\":%.2f}",
                    amount, escapeJson(recipient.getHolderName()), escapeJson(recipient.getAccountNumber()), sender.getBalance()
                ));
            } else {
                sendJsonResponse(exchange, 400, jsonResponse(false, "Transfer failed."));
            }
        }
    }

    /**
     * Handler: GET /api/statement?accountNumber=...
     */
    static class StatementHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            addCorsHeaders(exchange);
            if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(204, -1);
                return;
            }

            Map<String, String> queryParams = parseQueryParams(exchange.getRequestURI().getQuery());
            String accNo = queryParams.get("accountNumber");

            Account acc = bank.findAccount(accNo);
            if (acc == null) {
                sendJsonResponse(exchange, 404, jsonResponse(false, "Account not found."));
                return;
            }

            List<Transaction> txns = bank.getTransactionsForAccount(accNo);
            StringBuilder sb = new StringBuilder();
            sb.append("{\"success\":true,\"accountNumber\":\"").append(escapeJson(acc.getAccountNumber()))
              .append("\",\"balance\":").append(acc.getBalance())
              .append(",\"transactions\":[");
```

```
            for (int i = 0; i < txns.size(); i++) {
                Transaction t = txns.get(i);
                sb.append(String.format(
                    "{\"id\":\"%s\",\"type\":\"%s\",\"amount\":%.2f,\"date\":\"%s\",\"remarks\":\"%s\"}",
                    escapeJson(t.getTransactionId()),
                    escapeJson(t.getType()),
                    t.getAmount(),
                    escapeJson(t.getDate()),
                    escapeJson(t.getRemarks())
                ));
                if (i < txns.size() - 1) {
                    sb.append(",");
                }
            }
            sb.append("]}");

            sendJsonResponse(exchange, 200, sb.toString());
        }
    }

    // ================== STATIC FILE HANDLER ==================

    /**
     * Serves HTML, CSS, JS, and asset files from the BankWebUI folder.
     */
    static class StaticFileHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            String path = exchange.getRequestURI().getPath();

            // Normalize path: "/" -> "/index.html"
            if (path == null || path.equals("/") || path.trim().isEmpty()) {
                path = "/index.html";
            }

            // Remove leading slash
            if (path.startsWith("/")) {
                path = path.substring(1);
            }

            File requestedFile = new File(webUiDir, path);

            if (!requestedFile.exists() || requestedFile.isDirectory()) {
                // Return 404
                String notFound = "<html><body><h1>404 Not Found</h1><p>The requested file <code>" + path + "</code> was not
found.</p><a href='/'>Go to Home</a></body></html>";
                byte[] bytes = notFound.getBytes(StandardCharsets.UTF_8);
                exchange.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
                exchange.sendResponseHeaders(404, bytes.length);
                try (OutputStream os = exchange.getResponseBody()) {
                    os.write(bytes);
                }
                return;
            }

            // Detect MIME Type
            String mimeType = getMimeType(requestedFile.getName());
            exchange.getResponseHeaders().set("Content-Type", mimeType);
            exchange.getResponseHeaders().set("Cache-Control", "no-cache, no-store, must-revalidate");

            byte[] fileBytes = Files.readAllBytes(requestedFile.toPath());
            exchange.sendResponseHeaders(200, fileBytes.length);
            try (OutputStream os = exchange.getResponseBody()) {
                os.write(fileBytes);
            }
        }
    }
```

```
    // ================== HELPER METHODS ==================

    private static String getMimeType(String filename) {
        String lower = filename.toLowerCase();
        if (lower.endsWith(".html") || lower.endsWith(".htm")) return "text/html; charset=UTF-8";
        if (lower.endsWith(".css")) return "text/css; charset=UTF-8";
        if (lower.endsWith(".js")) return "application/javascript; charset=UTF-8";
        if (lower.endsWith(".json")) return "application/json; charset=UTF-8";
        if (lower.endsWith(".svg")) return "image/svg+xml";
        if (lower.endsWith(".png")) return "image/png";
        if (lower.endsWith(".jpg") || lower.endsWith(".jpeg")) return "image/jpeg";
        if (lower.endsWith(".ico")) return "image/x-icon";
        return "text/plain; charset=UTF-8";
    }

    private static void addCorsHeaders(HttpExchange exchange) {
        exchange.getResponseHeaders().set("Access-Control-Allow-Origin", "*");
        exchange.getResponseHeaders().set("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
        exchange.getResponseHeaders().set("Access-Control-Allow-Headers", "Content-Type, Authorization");
    }

    private static void sendJsonResponse(HttpExchange exchange, int statusCode, String json) throws IOException {
        exchange.getResponseHeaders().set("Content-Type", "application/json; charset=UTF-8");
        byte[] bytes = json.getBytes(StandardCharsets.UTF_8);
        exchange.sendResponseHeaders(statusCode, bytes.length);
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(bytes);
        }
    }

    private static String jsonResponse(boolean success, String message) {
        return "{\"success\":" + success + ",\"message\":\"" + escapeJson(message) + "\"}";
    }

    private static String escapeJson(String s) {
        if (s == null) return "";
        return s.replace("\\", "\\\\")
                .replace("\"", "\\\"")
                .replace("\b", "\\b")
                .replace("\f", "\\f")
                .replace("\n", "\\n")
                .replace("\r", "\\r")
                .replace("\t", "\\t");
    }

    private static Map<String, String> parseRequestBody(HttpExchange exchange) throws IOException {
        InputStream is = exchange.getRequestBody();
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        byte[] buffer = new byte[1024];
        int len;
        while ((len = is.read(buffer)) != -1) {
            baos.write(buffer, 0, len);
        }
        String body = baos.toString(StandardCharsets.UTF_8.name()).trim();
        Map<String, String> map = new HashMap<>();

        if (body.isEmpty()) return map;

        // If JSON format
        if (body.startsWith("{") && body.endsWith("}")) {
            String inner = body.substring(1, body.length() - 1);
            // Basic JSON key-value extraction for primitive fields
            String[] tokens = inner.split(",(?=(?:[^\"]*\"[^\"]*\")*[^\"]*$)");
            for (String token : tokens) {
                int colonIdx = token.indexOf(':');
                if (colonIdx > 0) {
                    String key = token.substring(0, colonIdx).trim().replace("\"", "");
                    String val = token.substring(colonIdx + 1).trim();
```

```
                    if (val.startsWith("\"") && val.endsWith("\"")) {
                        val = val.substring(1, val.length() - 1);
                    }
                    map.put(key, val);
                }
            }
        } else {
            // Form URL Encoded
            String[] pairs = body.split("&");
            for (String pair : pairs) {
                int eqIdx = pair.indexOf('=');
                if (eqIdx > 0) {
                    String key = URLDecoder.decode(pair.substring(0, eqIdx), "UTF-8");
                    String val = URLDecoder.decode(pair.substring(eqIdx + 1), "UTF-8");
                    map.put(key, val);
                }
            }
        }
        return map;
    }

    private static Map<String, String> parseQueryParams(String query) {
        Map<String, String> map = new HashMap<>();
        if (query == null || query.trim().isEmpty()) return map;
        try {
            String[] pairs = query.split("&");
            for (String pair : pairs) {
                int eqIdx = pair.indexOf('=');
                if (eqIdx > 0) {
                    String key = URLDecoder.decode(pair.substring(0, eqIdx), "UTF-8");
                    String val = URLDecoder.decode(pair.substring(eqIdx + 1), "UTF-8");
                    map.put(key, val);
                }
            }
        } catch (Exception e) {
            // Ignore malformed query string
        }
        return map;
    }

    private static double parseDouble(String str, double def) {
        if (str == null) return def;
        try {
            return Double.parseDouble(str.trim());
        } catch (NumberFormatException e) {
            return def;
        }
    }
}
```

2.1 index.html

Path: `BankWebUI/index.html`

HTML - Modern Banking Home & Landing Page

```
<!--
  index.html

  Purpose:
  Official Landing Page / Home Page for Apex National Bank Web System.
  Welcomes visitors, showcases key banking features, and provides direct links to
  Account Login and New Account Registration.

  Target Audience: 2nd-Year Engineering Students learning Web Design & Navigation.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Apex National Bank - Modern Online Banking</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Navigation Bar -->
    <nav class="navbar">
        <a href="index.html" class="brand">
            🏛️ Apex National Bank
        </a>
        <ul class="nav-links">
            <li><a href="index.html" class="active">Home</a></li>
            <li><a href="login.html">Login</a></li>
            <li><a href="register.html">Open Account</a></li>
        </ul>
    </nav>

    <!-- Hero / Main Section -->
    <main class="container" style="margin-top: 3rem;">
        <div class="card" style="text-align: center; padding: 3.5rem 2rem; background: linear-gradient(180deg, #ffffff 0%,
#f8fafc 100%);">
            <h1 style="font-size: 2.5rem; color: var(--primary-color); margin-bottom: 1rem;">
                Simple, Secure & Smart Banking
            </h1>
            <p style="font-size: 1.15rem; color: var(--text-muted); max-width: 650px; margin: 0 auto 2rem auto;">
                Welcome to Apex National Bank. Manage your money, transfer funds securely in real-time, deposit cash, and view
your complete transaction history from any device.
            </p>

            <div style="display: flex; justify-content: center; gap: 1rem; flex-wrap: wrap; margin-bottom: 2.5rem;">
                <a href="login.html" class="btn btn-primary" style="width: auto; padding: 0.85rem 2.25rem; font-size: 1.05rem;">
                    Sign In to Portal &rarr;
                </a>
                <a href="register.html" class="btn btn-outline" style="width: auto; padding: 0.85rem 2.25rem; font-size:
1.05rem;">
                    Open New Account
                </a>
            </div>

            <!-- Features Grid -->
            <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 1.5rem; text-align:
left; margin-top: 2rem;">
                <div style="padding: 1.5rem; background: white; border-radius: var(--radius-md); border: 1px solid var(--border-
color);">
                    <div style="font-size: 2rem; margin-bottom: 0.5rem;">⚡</div>
```

```
                    <h3 style="font-size: 1.1rem; margin-bottom: 0.4rem;">Instant Transfers</h3>
                    <p style="font-size: 0.85rem; color: var(--text-muted);">Transfer funds between accounts with instant balance
updates and verification.</p>
                </div>

                <div style="padding: 1.5rem; background: white; border-radius: var(--radius-md); border: 1px solid var(--border-
color);">
                    <div style="font-size: 2rem; margin-bottom: 0.5rem;">🔒</div>
                    <h3 style="font-size: 1.1rem; margin-bottom: 0.4rem;">PIN Protection</h3>
                    <p style="font-size: 0.85rem; color: var(--text-muted);">All debit and transfer operations require 4-digit
security PIN verification.</p>
                </div>

                <div style="padding: 1.5rem; background: white; border-radius: var(--radius-md); border: 1px solid var(--border-
color);">
                    <div style="font-size: 2rem; margin-bottom: 0.5rem;">📜</div>
                    <h3 style="font-size: 1.1rem; margin-bottom: 0.4rem;">Mini Statements</h3>
                    <p style="font-size: 0.85rem; color: var(--text-muted);">Detailed audit logs with timestamps, transaction
IDs, and contextual notes.</p>
                </div>

                <div style="padding: 1.5rem; background: white; border-radius: var(--radius-md); border: 1px solid var(--border-
color);">
                    <div style="font-size: 2rem; margin-bottom: 0.5rem;">💾</div>
                    <h3 style="font-size: 1.1rem; margin-bottom: 0.4rem;">Persistent Data</h3>
                    <p style="font-size: 0.85rem; color: var(--text-muted);">Data is automatically persisted to file storage via
the Core Java backend.</p>
                </div>
            </div>
        </div>
    </main>

    <!-- Footer -->
    <footer>
        &copy; 2026 Apex National Bank &bull; Powered by Java Core HttpServer & Web UI
    </footer>

</body>
</html>
```

2.2 login.html

Path: `BankWebUI/login.html`

HTML - Customer Authentication Login Form

```
<!--
  login.html

  Purpose:
  Interactive Bank Customer Login screen.
  Submits credentials to /api/login via app.js to authenticate against the Java backend.

  Target Audience: 2nd-Year Engineering Students learning Web Frontend & Java Backend Integration.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login - Apex National Bank</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Top Navigation -->
    <nav class="navbar">
        <a href="index.html" class="brand">
            🏛️ Apex National Bank
        </a>
        <ul class="nav-links">
            <li><a href="index.html">Home</a></li>
            <li><a href="login.html" class="active">Login</a></li>
            <li><a href="register.html">Open Account</a></li>
        </ul>
    </nav>

    <!-- Main Auth Form Container -->
    <main class="auth-container">
        <div class="card">
            <div class="card-header">
                <h2>Welcome Back</h2>
                <p>Sign in to access your Apex Bank account</p>
            </div>

            <!-- Login Form (Handled by app.js) -->
            <form id="loginForm">
                <!-- Account Number Field -->
                <div class="form-group">
                    <label for="accountNumber">Account Number</label>
                    <input
                        type="text"
                        id="accountNumber"
                        name="accountNumber"
                        class="form-control"
                        placeholder="e.g. 1001"
                        required
                        autocomplete="off"
                    >
                    <p class="form-help">Enter your registered account number</p>
                </div>

                <!-- PIN Field -->
                <div class="form-group">
                    <label for="pin">Security PIN</label>
                    <input
```

```
                        type="password"
                        id="pin"
                        name="pin"
                        class="form-control"
                        placeholder="••••"
                        required
                        maxlength="6"
                        autocomplete="off"
                    >
                    <p class="form-help">Your 4-digit secret PIN code</p>
                </div>

                <!-- Submit Button -->
                <button type="submit" class="btn btn-primary">Sign In to Account</button>
            </form>

            <div class="auth-footer">
                Don't have an account yet? <a href="register.html">Register here</a>
            </div>
        </div>
    </main>

    <!-- Footer -->
    <footer>
        &copy; 2026 Apex National Bank &bull; Powered by Java Core HttpServer & Web UI
    </footer>

    <!-- App Script -->
    <script src="app.js"></script>
</body>
</html>
```

2.3 register.html

Path: `BankWebUI/register.html`

HTML - New Account Registration Form

```
<!--
  register.html

  Purpose:
  Interactive New Account Registration screen.
  Submits new customer data to /api/register via app.js to create accounts in the Java backend.

  Target Audience: 2nd-Year Engineering Students learning Web Frontend & Java Backend Integration.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Open Account - Apex National Bank</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Top Navigation -->
    <nav class="navbar">
        <a href="index.html" class="brand">
            🏛️ Apex National Bank
        </a>
        <ul class="nav-links">
            <li><a href="index.html">Home</a></li>
            <li><a href="login.html">Login</a></li>
            <li><a href="register.html" class="active">Open Account</a></li>
        </ul>
    </nav>

    <!-- Main Registration Container -->
    <main class="auth-container">
        <div class="card">
            <div class="card-header">
                <h2>Open New Account</h2>
                <p>Register online to start banking with Apex</p>
            </div>

            <!-- Registration Form (Handled by app.js) -->
            <form id="registerForm">
                <!-- Full Name -->
                <div class="form-group">
                    <label for="holderName">Account Holder Full Name</label>
                    <input
                        type="text"
                        id="holderName"
                        name="holderName"
                        class="form-control"
                        placeholder="e.g. Alice Smith"
                        required
                        minlength="2"
                    >
                </div>

                <!-- Account Number -->
                <div class="form-group">
                    <label for="accountNumber">Desired Account Number</label>
                    <input
                        type="text"
```

```
                        id="accountNumber"
                        name="accountNumber"
                        class="form-control"
                        placeholder="e.g. 1003"
                        required
                        pattern="[0-9A-Za-z]+"
                    >
                    <p class="form-help">Unique numeric identifier (e.g. 1003)</p>
                </div>

                <!-- Initial Deposit -->
                <div class="form-group">
                    <label for="initialDeposit">Initial Deposit ($)</label>
                    <input
                        type="number"
                        id="initialDeposit"
                        name="initialDeposit"
                        class="form-control"
                        placeholder="500.00"
                        required
                        min="0"
                        step="0.01"
                        value="100.00"
                    >
                    <p class="form-help">Opening balance (must be non-negative)</p>
                </div>

                <!-- PIN -->
                <div class="form-group">
                    <label for="pin">Create 4-Digit PIN</label>
                    <input
                        type="password"
                        id="pin"
                        name="pin"
                        class="form-control"
                        placeholder="••••"
                        required
                        maxlength="6"
                        pattern="[0-9]{4,6}"
                    >
                    <p class="form-help">Your secret security PIN for withdrawals</p>
                </div>

                <!-- Submit Button -->
                <button type="submit" class="btn btn-primary">Create Account</button>
            </form>

            <div class="auth-footer">
                Already have an account? <a href="login.html">Sign in here</a>
            </div>
        </div>
    </main>

    <!-- Footer -->
    <footer>
        &copy; 2026 Apex National Bank &bull; Powered by Java Core HttpServer & Web UI
    </footer>

    <!-- App Script -->
    <script src="app.js"></script>
</body>
</html>
```

2.4 dashboard.html

Path: `BankWebUI/dashboard.html`

HTML - Customer Dashboard & Live Mini Statement

```
<!--
  dashboard.html

  Purpose:
  Interactive User Account Dashboard.
  Displays live account balance, customer profile, quick action triggers (Deposit, Withdraw, Transfer),
  and automatically renders dynamic mini statement transaction records fetched from /api/statement.

  Target Audience: 2nd-Year Engineering Students learning Dashboard Layouts & REST API Integration.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Account Dashboard - Apex National Bank</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Navigation Bar -->
    <nav class="navbar">
        <a href="dashboard.html" class="brand">
            🏛️ Apex National Bank
        </a>
        <ul class="nav-links">
            <li><a href="dashboard.html" class="active">Dashboard</a></li>
            <li><a href="transfer.html">Transfer Funds</a></li>
            <li><a href="#" class="logout-btn">Sign Out</a></li>
        </ul>
    </nav>

    <!-- Main Container -->
    <main class="container" id="dashboardView">
        <!-- Welcome Banner -->
        <div class="welcome-banner">
            <div>
                <h1>Hello, <span class="user-name">Customer</span> 👋</h1>
                <p>Welcome to your personal online banking dashboard</p>
            </div>
            <div>
                <a href="transfer.html" class="btn btn-primary" style="width: auto;">+ New Transfer</a>
            </div>
        </div>

        <!-- Dashboard Grid (Balance & Actions) -->
        <div class="dashboard-grid">
            <!-- Live Balance Card -->
            <div class="balance-card">
                <div class="label">Available Account Balance</div>
                <div class="amount" id="accountBalance">$0.00</div>
                <div class="acc-meta">
                    <strong>Account No:</strong> <span class="user-acc">Loading...</span> &bull; <strong>Status:</strong> Active
                </div>
            </div>

            <!-- Quick Actions Card -->
            <div class="actions-card">
                <h3>Quick Actions</h3>
                <div class="action-buttons">
```

```
                    <button class="btn btn-outline" onclick="handleQuickDeposit()">📥 Deposit</button>
                    <button class="btn btn-outline" onclick="handleQuickWithdraw()">🏧 Withdraw</button>
                    <a href="transfer.html" class="btn btn-outline">💸 Transfer</a>
                    <a href="#statement" class="btn btn-outline">📜 Statement</a>
                </div>
            </div>
        </div>

        <!-- Recent Transactions / Mini Statement -->
        <div class="card" id="statement">
            <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1rem;">
                <h3 style="font-size: 1.25rem;">Recent Account Activity (Mini Statement)</h3>
                <button class="btn btn-outline" style="width: auto; padding: 0.4rem 0.8rem; font-size: 0.8rem;"
onclick="initDashboard()">🔄 Refresh</button>
            </div>

            <div class="table-responsive">
                <table class="table">
                    <thead>
                        <tr>
                            <th>Transaction ID</th>
                            <th>Date & Time</th>
                            <th>Type</th>
                            <th>Amount</th>
                            <th>Remarks / Note</th>
                        </tr>
                    </thead>
                    <tbody id="statementTableBody">
                        <tr>
                            <td colspan="5" style="text-align: center; color: var(--text-muted); padding: 1.5rem;">
                                Loading transaction records...
                            </td>
                        </tr>
                    </tbody>
                </table>
            </div>
        </div>
    </main>

    <!-- Footer -->
    <footer>
        &copy; 2026 Apex National Bank &bull; Powered by Java Core HttpServer & Web UI
    </footer>

    <!-- App Script -->
    <script src="app.js"></script>
</body>
</html>
```

2.5 transfer.html

Path: `BankWebUI/transfer.html`

HTML - Inter-Account Fund Transfer Screen

```
<!--
  transfer.html

  Purpose:
  Interactive Fund Transfer screen.
  Submits fund transfer instructions to /api/transfer via app.js to update both
  sender and recipient account balances in the Java backend.

  Target Audience: 2nd-Year Engineering Students learning Form Validation & Async Transfers.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Fund Transfer - Apex National Bank</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Navigation Bar -->
    <nav class="navbar">
        <a href="dashboard.html" class="brand">
            🏛️ Apex National Bank
        </a>
        <ul class="nav-links">
            <li><a href="dashboard.html">Dashboard</a></li>
            <li><a href="transfer.html" class="active">Transfer Funds</a></li>
            <li><a href="#" class="logout-btn">Sign Out</a></li>
        </ul>
    </nav>

    <!-- Main Container -->
    <main class="auth-container" style="max-width: 500px;">
        <div class="card">
            <div class="card-header">
                <h2>Transfer Funds</h2>
                <p>Instantly send money to any registered bank account</p>
            </div>

            <!-- Transfer Form (Handled by app.js) -->
            <form id="transferForm">
                <!-- Sender Account (Read Only) -->
                <div class="form-group">
                    <label for="fromAccount">From Account (Source)</label>
                    <input
                        type="text"
                        id="fromAccount"
                        name="fromAccount"
                        class="form-control"
                        value="Loading..."
                        readonly
                    >
                </div>

                <!-- Recipient Account -->
                <div class="form-group">
                    <label for="toAccount">Beneficiary Account Number</label>
                    <input
                        type="text"
```

```
                    id="toAccount"
                    name="toAccount"
                    class="form-control"
                    placeholder="e.g. 1002"
                    required
                    pattern="[0-9A-Za-z]+"
                >
                <p class="form-help">Enter recipient's registered account number</p>
            </div>

            <!-- Transfer Amount -->
            <div class="form-group">
                <label for="amount">Amount ($)</label>
                <input
                    type="number"
                    id="amount"
                    name="amount"
                    class="form-control"
                    placeholder="100.00"
                    required
                    min="0.01"
                    step="0.01"
                >
            </div>

            <!-- Remarks / Purpose -->
            <div class="form-group">
                <label for="remarks">Transfer Remarks (Optional)</label>
                <input
                    type="text"
                    id="remarks"
                    name="remarks"
                    class="form-control"
                    placeholder="e.g. Project stipend, College fees"
                >
            </div>

            <!-- Security PIN -->
            <div class="form-group">
                <label for="pin">Authorize with Your Security PIN</label>
                <input
                    type="password"
                    id="pin"
                    name="pin"
                    class="form-control"
                    placeholder="••••"
                    required
                    maxlength="6"
                    pattern="[0-9]{4,6}"
                >
            </div>

            <!-- Action Buttons -->
            <div style="display: flex; gap: 1rem; margin-top: 1.5rem;">
                <a href="dashboard.html" class="btn btn-secondary" style="flex: 1;">Cancel</a>
                <button type="submit" class="btn btn-primary" style="flex: 2;">Confirm Transfer</button>
            </div>
        </form>
    </div>
</main>

<!-- Footer -->
<footer>
    &copy; 2026 Apex National Bank &bull; Powered by Java Core HttpServer & Web UI
</footer>

<!-- App Script -->
<script src="app.js"></script>
```

```
</body>
</html>
```

2.6 style.css

Path: `BankWebUI/style.css`

CSS - Responsive Modern Styling & Theme

```css
/*
 * style.css
 *
 * Purpose:
 * Shared stylesheet for the Bank Management System Web UI Mockup.
 * Provides clean, modern, and accessible styling without any external CSS frameworks.
 *
 * Target Audience: 2nd-Year Engineering Students learning HTML5 & CSS3 layout techniques.
 */

:root {
    --primary-color: #1e40af;       /* Classic trustworthy banking blue */
    --primary-hover: #1d4ed8;
    --primary-light: #eff6ff;
    --accent-color: #0d9488;        /* Teal accent */
    --bg-color: #f1f5f9;            /* Soft light gray background */
    --card-bg: #ffffff;
    --text-main: #0f172a;
    --text-muted: #64748b;
    --border-color: #cbd5e1;
    --border-focus: #3b82f6;
    --success-color: #16a34a;
    --danger-color: #dc2626;
    --shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
    --shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -2px rgba(0, 0, 0, 0.1);
    --radius-md: 8px;
    --radius-lg: 12px;
}

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
    background-color: var(--bg-color);
    color: var(--text-main);
    line-height: 1.5;
    min-height: 100vh;
    display: flex;
    flex-direction: column;
}

/* ==================== NAVIGATION BAR ==================== */
.navbar {
    background-color: var(--primary-color);
    color: #ffffff;
    padding: 1rem 2rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
    box-shadow: var(--shadow-md);
}

.navbar .brand {
    font-size: 1.25rem;
    font-weight: 700;
    display: flex;
```

```
    align-items: center;
    gap: 0.5rem;
    text-decoration: none;
    color: #ffffff;
}

.navbar .nav-links {
    list-style: none;
    display: flex;
    gap: 1.5rem;
}

.navbar .nav-links a {
    color: #e2e8f0;
    text-decoration: none;
    font-weight: 500;
    transition: color 0.2s ease;
}

.navbar .nav-links a:hover,
.navbar .nav-links a.active {
    color: #ffffff;
    text-decoration: underline;
}

/* ==================== PAGE CONTAINERS ==================== */
.container {
    max-width: 1000px;
    margin: 2rem auto;
    padding: 0 1rem;
    width: 100%;
    flex: 1;
}

.auth-container {
    max-width: 440px;
    margin: 3rem auto;
    padding: 0 1rem;
    width: 100%;
    flex: 1;
}

/* ==================== CARDS & FORMS ==================== */
.card {
    background: var(--card-bg);
    border-radius: var(--radius-lg);
    box-shadow: var(--shadow-md);
    padding: 2rem;
    border: 1px solid var(--border-color);
}

.card-header {
    margin-bottom: 1.5rem;
    text-align: center;
}

.card-header h2 {
    font-size: 1.5rem;
    color: var(--primary-color);
    margin-bottom: 0.25rem;
}

.card-header p {
    color: var(--text-muted);
    font-size: 0.9rem;
}

.form-group {
```

```
    margin-bottom: 1.25rem;
}

.form-group label {
    display: block;
    font-weight: 600;
    margin-bottom: 0.4rem;
    font-size: 0.9rem;
    color: var(--text-main);
}

.form-control {
    width: 100%;
    padding: 0.75rem 0.85rem;
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    font-size: 0.95rem;
    transition: border-color 0.2s, box-shadow 0.2s;
    outline: none;
}

.form-control:focus {
    border-color: var(--border-focus);
    box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.15);
}

.form-control[readonly] {
    background-color: #f8fafc;
    cursor: not-allowed;
}

.form-help {
    font-size: 0.8rem;
    color: var(--text-muted);
    margin-top: 0.3rem;
}

/* ================== BUTTONS ================== */
.btn {
    display: inline-block;
    width: 100%;
    padding: 0.75rem 1.25rem;
    font-size: 1rem;
    font-weight: 600;
    text-align: center;
    border-radius: var(--radius-md);
    border: none;
    cursor: pointer;
    text-decoration: none;
    transition: background-color 0.2s, transform 0.1s;
}

.btn-primary {
    background-color: var(--primary-color);
    color: #ffffff;
}

.btn-primary:hover {
    background-color: var(--primary-hover);
}

.btn-secondary {
    background-color: #e2e8f0;
    color: var(--text-main);
}

.btn-secondary:hover {
    background-color: #cbd5e1;
```

```
}

.btn-outline {
    background-color: transparent;
    border: 1px solid var(--border-color);
    color: var(--text-main);
}

.btn-outline:hover {
    background-color: var(--primary-light);
    border-color: var(--primary-color);
    color: var(--primary-color);
}

/* ==================== DASHBOARD SPECIFIC STYLES ==================== */
.welcome-banner {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 2rem;
}

.welcome-banner h1 {
    font-size: 1.75rem;
    color: var(--text-main);
}

.welcome-banner p {
    color: var(--text-muted);
}

.dashboard-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 1.5rem;
    margin-bottom: 2rem;
}

.balance-card {
    background: linear-gradient(135deg, var(--primary-color), #2563eb);
    color: #ffffff;
    padding: 1.75rem;
    border-radius: var(--radius-lg);
    box-shadow: var(--shadow-md);
}

.balance-card .label {
    font-size: 0.9rem;
    opacity: 0.85;
    text-transform: uppercase;
    letter-spacing: 0.05em;
}

.balance-card .amount {
    font-size: 2.25rem;
    font-weight: 700;
    margin: 0.5rem 0;
}

.balance-card .acc-meta {
    font-size: 0.85rem;
    opacity: 0.9;
}

.actions-card {
    background: var(--card-bg);
    border-radius: var(--radius-lg);
    padding: 1.75rem;
```

```
    box-shadow: var(--shadow-md);
    border: 1px solid var(--border-color);
    display: flex;
    flex-direction: column;
    justify-content: center;
}

.actions-card h3 {
    margin-bottom: 1rem;
    font-size: 1.1rem;
}

.action-buttons {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 0.75rem;
}

/* ================== TABLES ================== */
.table-responsive {
    overflow-x: auto;
    margin-top: 1rem;
}

.table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.9rem;
}

.table th, .table td {
    padding: 0.85rem 1rem;
    text-align: left;
    border-bottom: 1px solid var(--border-color);
}

.table th {
    background-color: #f8fafc;
    color: var(--text-muted);
    font-weight: 600;
    text-transform: uppercase;
    font-size: 0.75rem;
    letter-spacing: 0.05em;
}

.badge {
    display: inline-block;
    padding: 0.25rem 0.6rem;
    border-radius: 9999px;
    font-size: 0.75rem;
    font-weight: 600;
}

.badge-deposit {
    background-color: #dcfce7;
    color: var(--success-color);
}

.badge-withdraw {
    background-color: #fee2e2;
    color: var(--danger-color);
}

.badge-transfer {
    background-color: #e0e7ff;
    color: var(--primary-color);
}
```

```css
/* ==================== FOOTER & ALERTS ==================== */
.auth-footer {
    text-align: center;
    margin-top: 1.25rem;
    font-size: 0.9rem;
    color: var(--text-muted);
}

.auth-footer a {
    color: var(--primary-color);
    text-decoration: none;
    font-weight: 600;
}

.auth-footer a:hover {
    text-decoration: underline;
}

.alert-mock {
    background-color: #fef3c7;
    border-left: 4px solid #f59e0b;
    padding: 0.75rem 1rem;
    font-size: 0.85rem;
    color: #92400e;
    margin-bottom: 1.5rem;
    border-radius: 0 var(--radius-md) var(--radius-md) 0;
}

footer {
    text-align: center;
    padding: 1.5rem;
    color: var(--text-muted);
    font-size: 0.85rem;
    border-top: 1px solid var(--border-color);
    background-color: #ffffff;
    margin-top: auto;
}
```

2.7 app.js

Path: `BankWebUI/app.js`

JavaScript - REST API Integration, Session & UI Logic

```
/**
 * app.js
 *
 * Purpose:
 * Client-side JavaScript connecting the Bank Management Web UI to the
 * Java HttpServer REST API (/api/login, /api/register, /api/deposit, /api/withdraw, /api/transfer, /api/statement).
 * Manages user session state in sessionStorage and dynamically renders live data.
 *
 * Target Audience: 2nd-Year Engineering Students learning Asynchronous JavaScript & REST APIs.
 */

// Global API Base URL (Relative path works automatically with BankWebServer)
const API_BASE = '/api';

// ==================== SESSION MANAGEMENT ====================

function getCurrentUser() {
    const userStr = sessionStorage.getItem('apex_bank_user');
    if (!userStr) return null;
    try {
        return JSON.parse(userStr);
    } catch (e) {
        return null;
    }
}

function setCurrentUser(userData) {
    sessionStorage.setItem('apex_bank_user', JSON.stringify(userData));
}

function clearSession() {
    sessionStorage.removeItem('apex_bank_user');
}

function requireAuth() {
    const user = getCurrentUser();
    if (!user || !user.accountNumber) {
        window.location.href = 'login.html';
        return null;
    }
    return user;
}

// ==================== UI NOTIFICATIONS (TOASTS) ====================

function showToast(message, type = 'success') {
    let toastContainer = document.getElementById('toast-container');
    if (!toastContainer) {
        toastContainer = document.createElement('div');
        toastContainer.id = 'toast-container';
        toastContainer.style.cssText = `
            position: fixed;
            top: 20px;
            right: 20px;
            z-index: 9999;
            display: flex;
            flex-direction: column;
            gap: 10px;
        `;
        document.body.appendChild(toastContainer);
```

```
    }

    const toast = document.createElement('div');
    toast.className = `toast toast-${type}`;
    toast.style.cssText = `
        background-color: ${type === 'success' ? '#16a34a' : '#dc2626'};
        color: white;
        padding: 12px 20px;
        border-radius: 8px;
        box-shadow: 0 4px 12px rgba(0,0,0,0.15);
        font-size: 0.95rem;
        font-weight: 500;
        min-width: 250px;
        max-width: 380px;
        opacity: 0;
        transform: translateY(-10px);
        transition: all 0.3s ease;
    `;
    toast.innerText = message;
    toastContainer.appendChild(toast);

    // Fade in
    setTimeout(() => {
        toast.style.opacity = '1';
        toast.style.transform = 'translateY(0)';
    }, 10);

    // Auto dismiss
    setTimeout(() => {
        toast.style.opacity = '0';
        toast.style.transform = 'translateY(-10px)';
        setTimeout(() => toast.remove(), 300);
    }, 4000);
}

// ==================== GENERIC API HELPER ====================

async function apiRequest(endpoint, method = 'GET', bodyData = null) {
    const options = {
        method: method,
        headers: {
            'Content-Type': 'application/json'
        }
    };

    if (bodyData && (method === 'POST' || method === 'PUT')) {
        options.body = JSON.stringify(bodyData);
    }

    try {
        const response = await fetch(`${API_BASE}${endpoint}`, options);
        const data = await response.json();
        return { status: response.status, data: data };
    } catch (err) {
        console.error('API Error:', err);
        return {
            status: 500,
            data: { success: false, message: 'Server is offline or unreachable. Please ensure BankWebServer is running.' }
        };
    }
}

// ==================== PAGE-SPECIFIC INITIALIZERS ====================

document.addEventListener('DOMContentLoaded', () => {
    // 1. LOGIN PAGE
    const loginForm = document.getElementById('loginForm');
    if (loginForm) {
```

```
    loginForm.addEventListener('submit', async (e) => {
        e.preventDefault();
        const accNo = document.getElementById('accountNumber').value.trim();
        const pin = document.getElementById('pin').value.trim();
        const submitBtn = loginForm.querySelector('button[type="submit"]');

        submitBtn.disabled = true;
        submitBtn.innerText = 'Verifying...';

        const res = await apiRequest('/login', 'POST', { accountNumber: accNo, pin: pin });
        submitBtn.disabled = false;
        submitBtn.innerText = 'Sign In to Account';

        if (res.data && res.data.success) {
            // Store user session including PIN for authenticating subsequent actions
            setCurrentUser({
                accountNumber: res.data.accountNumber,
                holderName: res.data.holderName,
                balance: res.data.balance,
                pin: pin
            });
            showToast(`Welcome back, ${res.data.holderName}!`, 'success');
            setTimeout(() => {
                window.location.href = 'dashboard.html';
            }, 1000);
        } else {
            showToast(res.data.message || 'Login failed. Please check credentials.', 'error');
        }
    });
}

// 2. REGISTRATION PAGE
const registerForm = document.getElementById('registerForm');
if (registerForm) {
    registerForm.addEventListener('submit', async (e) => {
        e.preventDefault();
        const name = document.getElementById('holderName').value.trim();
        const accNo = document.getElementById('accountNumber').value.trim();
        const initialDeposit = parseFloat(document.getElementById('initialDeposit').value);
        const pin = document.getElementById('pin').value.trim();
        const submitBtn = registerForm.querySelector('button[type="submit"]');

        submitBtn.disabled = true;
        submitBtn.innerText = 'Opening Account...';

        const res = await apiRequest('/register', 'POST', {
            holderName: name,
            accountNumber: accNo,
            initialDeposit: initialDeposit,
            pin: pin
        });

        submitBtn.disabled = false;
        submitBtn.innerText = 'Create Account';

        if (res.data && res.data.success) {
            showToast('Account created successfully! Redirecting to login...', 'success');
            setTimeout(() => {
                window.location.href = 'login.html';
            }, 1500);
        } else {
            showToast(res.data.message || 'Registration failed.', 'error');
        }
    });
}

// 3. DASHBOARD PAGE
const dashboardContainer = document.getElementById('dashboardView');
```

```
    if (dashboardContainer) {
        initDashboard();
    }

    // 4. TRANSFER PAGE
    const transferForm = document.getElementById('transferForm');
    if (transferForm) {
        initTransferPage();
    }

    // 5. LOGOUT LINKS
    const logoutLinks = document.querySelectorAll('.logout-btn');
    logoutLinks.forEach(link => {
        link.addEventListener('click', (e) => {
            e.preventDefault();
            clearSession();
            window.location.href = 'login.html';
        });
    });
});

// ==================== DASHBOARD FUNCTIONS ====================

async function initDashboard() {
    const user = requireAuth();
    if (!user) return;

    // Display user profile info
    const nameElements = document.querySelectorAll('.user-name');
    nameElements.forEach(el => el.innerText = user.holderName);

    const accElements = document.querySelectorAll('.user-acc');
    accElements.forEach(el => el.innerText = user.accountNumber);

    await refreshDashboardData(user);
}

async function refreshDashboardData(user) {
    // 1. Fetch live balance
    const accRes = await apiRequest(`/account?accountNumber=${user.accountNumber}&pin=${user.pin}`, 'GET');
    if (accRes.data && accRes.data.success) {
        const balElement = document.getElementById('accountBalance');
        if (balElement) {
            balElement.innerText = `$${parseFloat(accRes.data.balance).toFixed(2)}`;
        }
        // Update session
        user.balance = accRes.data.balance;
        setCurrentUser(user);
    }

    // 2. Fetch mini statement
    const stmtRes = await apiRequest(`/statement?accountNumber=${user.accountNumber}`, 'GET');
    const tableBody = document.getElementById('statementTableBody');
    if (tableBody) {
        tableBody.innerHTML = '';

        if (stmtRes.data && stmtRes.data.transactions && stmtRes.data.transactions.length > 0) {
            // Reverse so recent are first
            const txns = [...stmtRes.data.transactions].reverse();
            txns.forEach(txn => {
                const tr = document.createElement('tr');
                let badgeClass = 'badge-deposit';
                let amountPrefix = '+$';
                let amountColor = 'var(--success-color)';

                if (txn.type.toLowerCase() === 'withdraw') {
                    badgeClass = 'badge-withdraw';
                    amountPrefix = '-$';
```

```
                    amountColor = 'var(--danger-color)';
                } else if (txn.type.toLowerCase() === 'transfer') {
                    badgeClass = 'badge-transfer';
                    amountPrefix = '-$';
                    amountColor = 'var(--primary-color)';
                }

                tr.innerHTML = `
                    <td><code>${txn.id}</code></td>
                    <td>${txn.date}</td>
                    <td><span class="badge ${badgeClass}">${txn.type}</span></td>
                    <td style="color: ${amountColor}; font-weight: 600;">${amountPrefix}${parseFloat(txn.amount).toFixed(2)}</td>
                    <td>${txn.remarks}</td>
                `;
                tableBody.appendChild(tr);
            });
        } else {
            tableBody.innerHTML = `<tr><td colspan="5" style="text-align: center; color: var(--text-muted); padding: 2rem;">No
transaction activity recorded yet.</td></tr>`;
        }
    }
}

// Modal / Prompt Deposit
async function handleQuickDeposit() {
    const user = requireAuth();
    if (!user) return;

    const amountStr = prompt(`Enter amount to deposit into Account ${user.accountNumber} ($):`);
    if (!amountStr) return;

    const amount = parseFloat(amountStr);
    if (isNaN(amount) || amount <= 0) {
        showToast('Please enter a valid positive deposit amount.', 'error');
        return;
    }

    const res = await apiRequest('/deposit', 'POST', {
        accountNumber: user.accountNumber,
        amount: amount
    });

    if (res.data && res.data.success) {
        showToast(res.data.message || `Successfully deposited $${amount.toFixed(2)}!`, 'success');
        await refreshDashboardData(user);
    } else {
        showToast(res.data.message || 'Deposit failed.', 'error');
    }
}

// Modal / Prompt Withdraw
async function handleQuickWithdraw() {
    const user = requireAuth();
    if (!user) return;

    const amountStr = prompt(`Enter amount to withdraw from Account ${user.accountNumber} ($):`);
    if (!amountStr) return;

    const amount = parseFloat(amountStr);
    if (isNaN(amount) || amount <= 0) {
        showToast('Please enter a valid positive withdrawal amount.', 'error');
        return;
    }

    const pin = prompt('Enter your 4-digit PIN to authorize withdrawal:');
    if (!pin) return;

    const res = await apiRequest('/withdraw', 'POST', {
```

```
        accountNumber: user.accountNumber,
        pin: pin,
        amount: amount
    });

    if (res.data && res.data.success) {
        showToast(res.data.message || `Successfully withdrew $${amount.toFixed(2)}!`, 'success');
        await refreshDashboardData(user);
    } else {
        showToast(res.data.message || 'Withdrawal failed.', 'error');
    }
}

// ==================== TRANSFER PAGE FUNCTIONS ====================

function initTransferPage() {
    const user = requireAuth();
    if (!user) return;

    const fromAccInput = document.getElementById('fromAccount');
    if (fromAccInput) {
        fromAccInput.value = `${user.accountNumber} (${user.holderName} - Balance: $${parseFloat(user.balance).toFixed(2)})`;
    }

    const transferForm = document.getElementById('transferForm');
    transferForm.addEventListener('submit', async (e) => {
        e.preventDefault();
        const toAcc = document.getElementById('toAccount').value.trim();
        const amount = parseFloat(document.getElementById('amount').value);
        const remarks = document.getElementById('remarks').value.trim();
        const pin = document.getElementById('pin').value.trim();
        const submitBtn = transferForm.querySelector('button[type="submit"]');

        submitBtn.disabled = true;
        submitBtn.innerText = 'Processing Transfer...';

        const res = await apiRequest('/transfer', 'POST', {
            fromAccount: user.accountNumber,
            pin: pin,
            toAccount: toAcc,
            amount: amount,
            remarks: remarks
        });

        submitBtn.disabled = false;
        submitBtn.innerText = 'Confirm Transfer';

        if (res.data && res.data.success) {
            showToast(res.data.message || 'Transfer completed successfully!', 'success');
            setTimeout(() => {
                window.location.href = 'dashboard.html';
            }, 1500);
        } else {
            showToast(res.data.message || 'Transfer failed. Check details.', 'error');
        }
    });
}
```

3.1 run-server.bat

Path: `run-server.bat`

Batch - One-Click Launcher for Web Server & UI

```
@echo off
title Apex Bank Web Server
echo ================================================
echo       APEX BANK MANAGEMENT SYSTEM WEB SERVER
echo ================================================
echo.

cd /d "%~dp0BankJavaApp"

echo [1/3] Compiling Java classes...
javac *.java
if %errorlevel% neq 0 (
    echo [ERROR] Java compilation failed! Please check your JDK installation.
    pause
    exit /b %errorlevel%
)

echo [2/3] Starting Java HTTP Web Server on port 8080...
echo.
echo ================================================
echo   Server is running at: http://localhost:8080
echo   Press Ctrl + C to stop the server anytime.
echo ================================================
echo.

:: Automatically open the web browser after a brief delay
start "" "http://localhost:8080"

:: Launch the Java HTTP Server
java BankWebServer

pause
```

3.2 accounts.txt

Path: `BankJavaApp/accounts.txt`

Text - Comma-Separated Database File

```
1001,Alice Smith,1234,1000.00
1002,Bob Johnson,5678,800.00
0000,pot yu,0000,712.00
2001,Charlie Brown,9999,700.00
1234,alok yadav,0000,88.00
```

3.3 README.md

Path: `README.md`

Markdown - Comprehensive Project Guide & Setup Instructions

````
# 🏦 Bank Management System (Java Full-Stack Website & Console CLI)

A clean, beginner-friendly Bank Management System designed specifically for **second-year engineering students** learning **Core Java, Object-Oriented Programming (OOP), and Web Fundamentals (HTML5, CSS3, JavaScript & REST APIs)**.

No heavy frameworks (no Spring Boot, no Maven/Gradle, no external database dependencies) — built 100% with **Standard Core Java (`com.sun.net.httpserver.HttpServer`)** and **pure HTML/CSS/JavaScript**.

---

## 📂 Project Directory Structure

```text
BankManagementSystem/
├── run-server.bat                # 🚀 One-click Windows launcher for Web Server & UI
│
├── BankJavaApp/                  # Java Backend & CLI
│   ├── BankWebServer.java        # Java HTTP Web Server (port 8080) serving REST APIs & UI
│   ├── Main.java                 # Menu-driven console interface (CLI)
│   ├── Bank.java                 # Business logic engine (validations, transactions, accounts)
│   ├── Account.java              # Account entity model
│   ├── Transaction.java          # Transaction history entity model
│   ├── FileHandler.java          # Persistence layer (reads & writes accounts.txt)
│   └── accounts.txt              # Comma-separated text database
│
├── BankWebUI/                    # Dynamic Web Frontend
│   ├── index.html                # Official Homepage / Landing page
│   ├── login.html                # Account Login page
│   ├── register.html             # New Account Registration page
│   ├── dashboard.html            # Customer Dashboard (live balance, deposit, withdraw, statements)
│   ├── transfer.html             # Inter-account fund transfer screen
│   ├── style.css                 # Shared responsive modern CSS stylesheet
│   └── app.js                    # Frontend JavaScript connecting UI to Java REST APIs
│
└── README.md                     # Documentation & setup guide
```

---

## 🚀 Quick Start: Running & Viewing from Other PCs / Devices

### 🌐 Access Online from ANY PC or Mobile Phone (Worldwide)
Your website is currently deployed live on a public HTTPS tunnel:
👉 **[https://local-recordings-which-richmond.trycloudflare.com](https://local-recordings-which-richmond.trycloudflare.com)**

### 📶 Access from Devices on the Same Wi-Fi / Local Network
If your other PC or phone is connected to the same Wi-Fi router, open:
👉 **`http://192.168.0.109:8080`**

### ⚡ One-Click Online Deployment Launcher (Windows)
Double-click **`share-online.bat`** in the project root directory. It will automatically:
1. Compile and launch the Java Web Server.
2. Start Cloudflare Tunnel and generate a live public HTTPS link.

### 💻 Localhost Access (This PC Only)
Double-click **`run-server.bat`** or run:
```powershell
cd "C:\Users\HP\.gemini\antigravity\scratch\BankManagementSystem\BankJavaApp"
javac *.java
java BankWebServer
````
````

```
Then open **[http://localhost:8080](http://localhost:8080)** in your browser.

---

## 💻 Features Available on the Website

1. **🏠 Landing Page (`index.html`)**: Feature highlights and quick links to open accounts or sign in.
2. **📝 Open New Account (`register.html`)**: Register a new account with Full Name, Account Number, Initial Deposit, and 4-digit PIN.
3. **🔑 Secure Login (`login.html`)**: Authenticate with Account Number and PIN.
4. **📊 Interactive Dashboard (`dashboard.html`)**:
   - Live balance display.
   - **📥 Quick Deposit**: Add money to your account with immediate balance refresh.
   - **🏧 Quick Withdraw**: Secure cash withdrawal verified with your PIN.
   - **📜 Live Mini Statement**: Auto-rendered table of all recent transactions (Deposits, Withdrawals, Transfers).
5. **💸 Fund Transfer (`transfer.html`)**: Transfer funds to another account number with security PIN authorization.
6. **💾 Auto-Persistence**: Any action taken on the web interface is instantly saved into `accounts.txt` by `FileHandler.java`.

---

## 🔌 Java REST API Endpoints

The built-in Java server (`BankWebServer.java`) exposes the following endpoints:

| Method | Endpoint | Description | Sample Payload / Query |
|---|---|---|---|
| `POST` | `/api/register` | Open a new bank account | `{"accountNumber":"1001", "holderName":"Alice", "pin":"1234", "initialDeposit":500}` |
| `POST` | `/api/login` | Authenticate customer | `{"accountNumber":"1001", "pin":"1234"}` |
| `GET` | `/api/account` | Retrieve account details | `?accountNumber=1001&pin=1234` |
| `POST` | `/api/deposit` | Deposit money | `{"accountNumber":"1001", "amount":200}` |
| `POST` | `/api/withdraw` | Withdraw money (PIN required) | `{"accountNumber":"1001", "pin":"1234", "amount":100}` |
| `POST` | `/api/transfer` | Transfer funds to another account | `{"fromAccount":"1001", "pin":"1234", "toAccount":"1002", "amount":50, "remarks":"Rent"}` |
| `GET` | `/api/statement` | Get transaction history list | `?accountNumber=1001` |

---

## ☕ Alternative: Running the Console Application (CLI)

If you wish to run the console menu-driven version:
```powershell
cd "C:\Users\HP\.gemini\antigravity\scratch\BankManagementSystem\BankJavaApp"
javac *.java
java Main
```

🎓 Key Engineering Learning Points

1. **Full-Stack Architecture**: Demonstrates the separation between the **Frontend (HTML/CSS/JS)** and the **Backend (Java HTTP Server & Business Logic)**.
2. **REST API Design**: Shows how HTTP request methods (`GET`, `POST`), status codes (`200`, `400`, `401`, `404`), and JSON payloads coordinate client-server communication.
3. **Data Persistence**: Illustrates file I/O streams (`BufferedReader`, `BufferedWriter`) in Java.
4. **Security & Validation**: Enforces credential authentication, balance boundary checks, and input sanitization.

📜 License
This project is open-source and intended for academic study and coursework submission.
```